Lab2A - Introduction to PyTorch

Tensor (`torch.tensor`) is the data structure used in PyTorch to build a deep learning system. Tensors are similar to NumPy's `ndarrays` , with the addition being that Tensors can also be used on a GPU to accelerate computing.

Objectives:

In this lab, you learn how to

- Create tensors in PyTorch
- Perform mathematical operation on tensors
- Convert between PyTorch tensor and Numpy array
- Reshape a PyTorch tensor
- Transfer tensor to and from GPU

Table of Content:

1. Creating tensors
2. Tensor operations
3. Indexing
4. Reshaping tensors
5. CUDA Tensors
6. Exercise

Reference:

- PyTorch Official Tutorial: What is PyTorch

---

1. Creating tensors

```
import torch
```

Construct a 5x3 matrix, uninitialized

```
x = torch.empty(5, 3)
print(x)
```

```
tensor([[1.0661e+25, 3.0787e-41, 1.0000e+00],
        [1.0000e+00, 1.1975e+24, 3.0787e-41],
        [1.1974e+24, 3.0787e-41, 1.1969e+24],
        [3.0787e-41, 1.1974e+24, 3.0787e-41],
        [1.1975e+24, 3.0787e-41, 1.1645e-20]])
```

Construct a tensor filled with random numbers from a uniform distribution on the interval `[0, 1)` .

```
x = torch.rand(5, 3)
print(x)
```

```
tensor([[0.8215, 0.7875, 0.0379],
        [0.6684, 0.2378, 0.5110],
        [0.4316, 0.1092, 0.0263],
        [0.1296, 0.9684, 0.0315],
        [0.0876, 0.1452, 0.8343]])
```

Construct a tensor filled with random numbers from a normal distribution with mean 0 and variance 1.

```
x = torch.randn(5, 3)
print(x)
```

```
tensor([[-0.3664,  0.2633, -0.5742],
        [-0.8448,  1.2114, -0.2034],
        [ 0.9845, -0.7810, -1.1963],
        [-0.1103, -0.3548,  0.8668],
        [ 0.3946,  1.5033, -0.2965]])
```

Construct a matrix filled with zeros and of dtype long

```
x = torch.zeros(5, 3, dtype=torch.long)
print(x)
```

```
tensor([[0, 0, 0],
        [0, 0, 0],
        [0, 0, 0],
        [0, 0, 0],
        [0, 0, 0]])
```

```
x = torch.ones(5, 3)
print(x)
```

```
tensor([[1., 1., 1.],
        [1., 1., 1.],
        [1., 1., 1.],
        [1., 1., 1.],
        [1., 1., 1.]])
```

---

2. Tensor Operations

Size of tensors

```
x = torch.rand(5, 3)
print(x)
print(x.size())   # torch.Size is actually a tuple
print(x.shape)
```

```
tensor([[0.3134, 0.9470, 0.4261],
        [0.0155, 0.0150, 0.3553],
        [0.3981, 0.3930, 0.4040],
        [0.9707, 0.3812, 0.3539],
        [0.9843, 0.1122, 0.7694]])
torch.Size([5, 3])
torch.Size([5, 3])
```

Addition

There are multiple syntaxes for operations.

Addition: syntax 1

```
x = torch.rand(3, 2)
print('x:\n', x)
y = torch.rand(3, 2)
print('y:\n', y)
```

```
z = x + y
print('x+y:\n', z)
```

```
x:
 tensor([[0.0776, 0.5578],
        [0.9773, 0.8880],
        [0.9000, 0.5249]])
y:
 tensor([[0.5977, 0.6239],
        [0.0501, 0.0323],
        [0.3845, 0.4517]])
x+y:
 tensor([[0.6753, 1.1818],
        [1.0273, 0.9203],
        [1.2845, 0.9765]])
```

Addition: syntax 2

```
z = torch.add(x, y)
print('x+y:\n', z)
```

```
x+y:
 tensor([[0.6753, 1.1818],
        [1.0273, 0.9203],
        [1.2845, 0.9765]])
```

Addition: syntax 3 (in-place)

- Any operation that mutates a tensor in-place is post-fixed with an `_`. For example: `x.copy_(y)`, `x.t_()`, will change `x`.

```
print('x\n', x)
print('y\n', y)
```

```
x
 tensor([[0.0776, 0.5578],
        [0.9773, 0.8880],
        [0.9000, 0.5249]])
y
 tensor([[0.5977, 0.6239],
        [0.0501, 0.0323],
        [0.3845, 0.4517]])
```

```
y.add_(x)
print(y)
```

```
tensor([[0.6753, 1.1818],
        [1.0273, 0.9203],
        [1.2845, 0.9765]])
```

Multiplication

Different from numpy which uses mainly `dot` to perform different types of matrix multiplication, PyTorch uses different commands for vector-vector multiplication (`dot`), matrix-vector multiplication (`mv`) and matrix-matrix multiplication (`mm`)

dot

```
a = torch.Tensor([4, 2])
b = torch.Tensor([3, 1])
r = torch.dot(a, b)

print(r)
```

```
tensor(14.)
```

mv

```
mat = torch.randn(2, 4)
vec = torch.randn(4)
r = torch.mv(mat, vec)
print(r)
```

```
tensor([1.0645, 1.1527])
```

mm

```
mat1 = torch.randn(2, 3)
mat2 = torch.randn(3, 4)
r = torch.mm(mat1, mat2)

print(r)
```

```
tensor([[-0.2309, -0.7671, -0.6623, -0.2645],
        [ 0.5864,  1.3051,  0.3537, -0.8557]])
```

3. Indexing

You can use standard Numpy-like indexing with Torch

```
x = torch.randint(0, 100, (5,10))
print(x)
```

```
tensor([[ 8, 52, 17, 76, 40, 44, 13, 17, 18, 60],
        [78, 21, 58, 88, 74, 24,  7, 12, 55, 95],
        [56, 22,  2, 50, 71, 59,  7, 33, 26, 92],
        [91, 28, 59,  3, 66, 79, 30, 68, 15, 21],
        [31, 58, 36, 30, 98, 53, 24, 74, 36,  3]])
```

```
# accessing column 1
print(x[:,1])
```

```
tensor([52, 21, 22, 28, 58])
```

```
# accessing columns 2 and 3
print(x[:, 2:4])
```

```
tensor([[17, 76],
        [58, 88],
        [ 2, 50],
        [59,  3],
        [36, 30]])
```

```
# accessing row 1
print(x[1,:])
```

```
tensor([78, 21, 58, 88, 74, 24,  7, 12, 55, 95])
```

```
# accessing rows 2 and 3
print(x[2:4,:])
```

```
tensor([[56, 22,  2, 50, 71, 59,  7, 33, 26, 92],
        [91, 28, 59,  3, 66, 79, 30, 68, 15, 21]])
```

4. Reshaping Tensors

`Tensor.reshape`

Returns a tensor with the same data and number of elements as self but with the specified shape.

```
x = torch.randint(0, 100, (2,4))
print('x:\n', x)
```

```
x:
 tensor([[33, 11, 64, 84],
        [41, 49, 35, 41]])
```

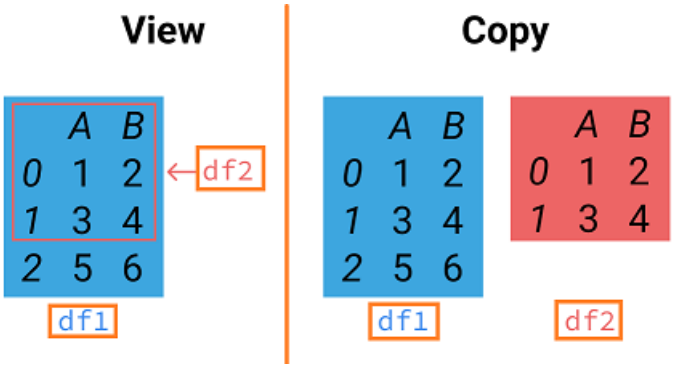
```
# Reshape from (2, 4) to (8, 1)
y = x.reshape(8, -1)
print('y:\n', y)
```

```
y:
 tensor([[33],
        [11],
        [64],
        [84],
        [41],
        [49],
        [35],
        [41]])
```

```
# Reshape from (2, 4) to (4, 2)
z = x.reshape(4, 2)
print('z:\n', z)
```

```
z:
 tensor([[33, 11],
        [64, 84],
        [41, 49],
        [35, 41]])
```

This method returns a **view** if shape is compatible with the current shape. Else, it may return a **copy**. This allows it to work with both contiguous and non-contigous data.



In the examples above, we create a view since the shapes of reshaped tensors `y` and `z` are compatible with the original tensor `x`. Note that after a change is performed on `x`, then the changes will occur to both `y` and `z`.

The following code confirms that `y` and `z` are indeed **views** of `x`. Any changes to `x` will be observed in `y` and `z` as well.

```
x[0,0] = -3

print('x:\n', x)
print('y:\n', y)
print('z:\n', z)
```

```
x:
 tensor([[-3, 11, 64, 84],
        [41, 49, 35, 41]])
y:
 tensor([[-3],
        [11],
        [64],
        [84],
        [41],
        [49],
        [35],
        [41]])
z:
 tensor([[-3, 11],
        [64, 84],
        [41, 49],
        [35, 41]])
```

`Tensor.view`

`Tensor.view` always returns a **view** of the original tensor with the new shape, i.e., it will share the underlying data with the original tensor.

```
x = torch.randint(0, 100, (2, 4))
print('x:\n', x)
```

```
x:
 tensor([[32, 87, 14, 23],
        [34, 39,  5, 60]])
```

```
# Convert from (2, 4) to (8, 1)
y = x.view(8, -1)
print('y:\n', y)

```

```
y:
 tensor([[32],
        [87],
        [14],
        [23],
        [34],
        [39],
        [ 5],
        [60]])
```

```
# Convert from (2, 4) to (4, 2)
z = x.view(4, 2)
print('z:\n', z)
```

```
z:
 tensor([[32, 87],
        [14, 23],
        [34, 39],
        [ 5, 60]])
```

Similar to the numpy's `reshape` function, pytorch's `view` returns a reference of the original matrix albeit in a different shape

5. CUDA Tensors

Creating tensor in the GPU

```
if torch.cuda.is_available():
    device = torch.device("cuda")  # define a cuda device
    y = torch.ones((2, 4), device = device) # Create the tensor in the GPU
    print(y)
```

```
tensor([[1., 1., 1., 1.],
        [1., 1., 1., 1.]], device='cuda:0')
```

```
if torch.cuda.is_available():
    y = torch.ones((2, 4), device = "cuda")  # Create the tensor in the GPU
    print(y)
```

```
tensor([[1., 1., 1., 1.],
        [1., 1., 1., 1.]], device='cuda:0')
```

Creating tensor in the cpu explicitly (default)

```
if torch.cuda.is_available():
    y = torch.ones_like(x, device = "cpu")      # create a tensor in the CPU
    print(y)
```

```
tensor([[1, 1, 1, 1],
        [1, 1, 1, 1]])
```

Transfering tensor from cpu to gpu

Transfer using the `.cuda()` command.

```
x = torch.rand(3, 2)  # create tensor in the CPU
print(x)

x = x.cuda()  # move to GPU
print(x)
```

```
tensor([[0.9413, 0.7052],
        [0.6504, 0.7185],
        [0.9437, 0.9174]])
tensor([[0.9413, 0.7052],
        [0.6504, 0.7185],
        [0.9437, 0.9174]], device='cuda:0')
```

Transfer using the `.to()` command

```
x = torch.rand(3, 2) # create tensor in the CPU
print(x)

device = torch.device('cuda')  # move to GPU
x = x.to(device)
print(x)
```

```
tensor([[0.8021, 0.7112],
        [0.1591, 0.7165],
        [0.7395, 0.1636]])
```

```
tensor([[0.8021, 0.7112],
        [0.1591, 0.7165],
        [0.7395, 0.1636]], device='cuda:0')
```

Transfering tensor from gpu to cpu

Transfer using the `.cpu()` command.

```
x = torch.rand(3, 2, device = 'cuda')  # create tensor in the GPU
print(x)

x = x.cpu()  # move to CPU
print(x)
```

```
tensor([[0.5989, 0.7337],
        [0.6643, 0.6747],
        [0.5550, 0.9232]], device='cuda:0')
tensor([[0.5989, 0.7337],
        [0.6643, 0.6747],
        [0.5550, 0.9232]])
```

Transfer using the `.to()` command.

```
x = torch.rand(3, 2, device = 'cuda')  # create tensor in the GPU
print(x)

device = torch.device('cpu')
x = x.to(device)  # move to CPU
print(x)
```

```
tensor([[0.6067, 0.4409],
        [0.0962, 0.8024],
        [0.3449, 0.6410]], device='cuda:0')
tensor([[0.6067, 0.4409],
        [0.0962, 0.8024],
        [0.3449, 0.6410]])
```

Exercise

Question 1. The following code is used to preprocess a batch data for Logistic Regression.

1.1 Create a random tensor `X_ori` using the normal distribution of shape `(4, 16, 16, 3)`. The tensor represent `m=4` color image samples, each having a resolution of `(16, 16)` Expected ans: `Shape of X_ori: torch.Size([4, 16, 16, 3])`

```
# ...
print('Shape of X_ori:', X_ori.shape)
```

1.2 Reshape `X_ori` into a shape of `(4, 16*16*3)`. Then transpose the result to get a tensor of shape `(768, 4)` where each column represents a sample. Save the result as `X`.

```
Expected ans:
```
Shape of X: torch.Size([768, 4])
```
```

```
# ...
print('Shape of X:', X.shape)
```

1.3 Check if a GPU is available in the system. If yes, transfer the tensor `X` to the GPU. Then, print out the device ID of the GPU (`X.get_device()`).

```
Expected ans:
```
X is loaded to GPU: 0
```
```
...
```

---

**Question 2.**

2.1 Create the tensor `A`. Ensure that the datatype for `A` is `float32`:

```
A = [[3, 2, 4, 6],
     [2, 4, 2, 2],
     [5, 1, 2, 1]]
```

```
# ...
```

2.2 Extract the 2nd row from A. (Expected ans: `tensor([2., 4., 2., 2.])`)

```
# ...
```

2.3 Extract the 3rd column from A. (Expected ans: `tensor([4., 2., 2.])`)

```
# ...
```

2.4 Write the code to extract the following sub-block (rows 1 to 2 and columns 1 to 2) from A.

```
tensor([[4., 2.],
        [1., 2.]])
```

```
# ...
```

2.5 Compute the mean of all rows. You may need to convert the tensor to a double first. Note that in the computational graph, when you create

```
Expected ans:
```

```
tensor([3.7500, 2.5000, 2.2500], dtype=torch.float64)
```

```
# ...
```
```

```
# ...
```

2.6 Repeat question 2.5, but this time retain the original dimensions such that the output has a shape of (3,1)

```
Expected ans:


tensor([[3.7500],
        [2.5000],
        [2.2500]], dtype=torch.float64)
```

```
# ...
```

--- END OF LAB02A ---